

All You Can Cache: fútbol chileno

Informe experimental de Sistemas Distribuidos

Ricardo Lepin

Pablo Muñoz

12 de septiembre de 2026

Estado: documento de trabajo. Grupo 4, sección 2. El video está en preparación; incorporar su enlace y confirmar condiciones de entrega antes de enviar. Los resultados incluidos provienen de archivos JSON validados; una matriz incompleta no se presenta como terminada.

1. Problema y arquitectura

El sistema responde consultas de fútbol chileno con cache-aside: generador $\rightarrow$ servicio de caché $\rightarrow$ Redis; ante un miss consulta al scraper Soccerway y guarda un JSON con TTL. El cuarto servicio recoge métricas. Docker Compose separa dependencias y permite reproducir el despliegue. No hay una base de datos persistente: el término tiempo de origen corresponde aquí al scraping externo.

Para abordar la precarga indicada por la guía, se implementa también un modo preloaded: un snapshot real se obtiene antes de medir y se carga íntegramente en memoria al iniciar el scraper. Las consultas usan esas páginas sin llamadas externas; fuera del snapshot devuelven un error explícito. El alcance predeterminado cubre el catálogo funcional y los H2H de TTL, no todo el historial de Soccerway. No se mezcla su latencia de procesamiento en memoria con scraping en vivo. Python/FastAPI simplifican contratos HTTP y validación; Playwright permite obtener contenido dinámico; BeautifulSoup estructura el HTML; Redis ofrece TTL y políticas configurables sin reinventar almacenamiento.

Q1 entrega próximos partidos con fecha, hora, local y visita; Q2 resultados recientes; Q3 enfrentamientos entre dos equipos; Q4 partidos de un intervalo; Q5 tabla de posiciones. Las claves canónicas evitan que invertir los equipos en Q3 duplique respuestas. Se validan parámetros y estructura del origen; una respuesta de error no se guarda como dato válido. La extracción dinámica usa Playwright y BeautifulSoup, con límites de espera y cierre del navegador.

Redis admite allkeys-lru y allkeys-lfu, políticas aproximadas. LRU favorece recencia; LFU favorece frecuencia acumulada con envejecimiento. TTL reduce obsolescencia, pero un TTL demasiado corto añade misses incluso cuando hay capacidad. Un TTL largo mejora reutilización a costa de frescura, especialmente relevante para próximos partidos y tabla en días de juego.

2. Metodología y métricas

El catálogo funcional tiene 54 consultas estables: 16 Q1, 16 Q2, una Q3, 20 Q4 y una Q5. Se conservan semilla 42 y permutación al cambiar política o tamaño. Uniforme asigna probabilidad $1/N$; Zipf finita asigna $p_i = i^{-\alpha} / \sum_{j=1}^N j^{-\alpha}$. La selección afecta consultas completas, no parámetros aleatorios diferentes en cada repetición.

Las llegadas son de ciclo cerrado: se espera la respuesta y después el intervalo. Por tanto, el intervalo no es una tasa abierta independiente de la latencia. Cada corrida reinicia Redis y las métricas. La matriz real usa 40 consultas por configuración; es una muestra exploratoria, sin intervalos de confianza. Se recomienda repetir con distintas semillas antes de generalizar.

Las configuraciones funcionales admiten intervalos Uniformes entre cero y dos veces el promedio, o Zipf finita sobre 50 intervalos discretos normalizados al mismo promedio, con alpha 1.5. Esto permite llegadas dispersas o concentradas conservando la espera esperada. La matriz de presión mantiene intervalos constantes para aislar concentración de claves; no es un experimento de carga abierta concurrente.

La matriz sintética usa 1000 solicitudes, 1000 claves y valores de 64 KiB, TTL 3600 s, Zipf $\alpha = 1,1$ o Uniforme. Omite deliberadamente Soccerway para aislar presión de capacidad. No se usa para estimar scraping ni eficiencia relativa al origen. Redis incluye memoria interna en maxmemory; datos útiles y capacidad configurada no son equivalentes.

Hit rate = $H/(H + M)$; errores se informan por separado. Throughput es el número de respuestas exitosas dividido por la duración del cliente. Latencia p50/p95 se mide desde el cliente e incluye red, serialización y envío de telemetría; la latencia interna se conserva separadamente. Tiempo de scraping se mide sólo en accesos reales al origen. Evictions/min se calcula con la diferencia del contador Redis y duración. Expired keys no se suman a evictions.

La eficiencia solicitada se implementa como $(\sum t_{hit} - \sum t_{origen,miss})/(H + M + E)$, equivalente a la fórmula con tiempos promedio por clase. Es un indicador definido por la guía, no ahorro de tiempo contrafactual: puede resultar negativo. Los contadores acumulados se conservan y los percentiles/eventos se limitan a las 10000 observaciones más recientes. El runner verifica concordancia cliente-servidor y código de salida; conserva evidencia fallida sin incluirla entre éxitos.

3. Resultados medidos

3.1. TTL por consulta

Corridas válidas disponibles: 10.

Caso	Hits	Misses	Hit rate	p95 ms	Evict.	Exp.
q1-3s	0	2	0.000	4288.5	0	1
q1-60s	1	1	0.500	3468.4	0	0
q2-3s	0	2	0.000	3241.1	0	1
q2-60s	1	1	0.500	3251.0	0	0
q3-3s	0	2	0.000	10384.0	0	1
q3-60s	1	1	0.500	9230.4	0	0
q4-3s	0	2	0.000	5661.2	0	1
q4-60s	1	1	0.500	4948.7	0	0
q5-3s	0	2	0.000	3879.1	0	1
q5-60s	1	1	0.500	4148.8	0	0

Caso	Resp./s	p50 ms	Origen ms	Efic. ms	Evict./min
q1-3s	0.18	3059.8	3662.2	-3662.2	0.0
q1-60s	0.27	8.9	3456.0	-1727.8	0.0
q2-3s	0.19	3162.8	3189.7	-3189.7	0.0
q2-60s	0.28	10.2	3238.4	-1619.0	0.0
q3-3s	0.09	9040.4	9701.0	-9701.0	0.0
q3-60s	0.15	11.0	9218.0	-4608.8	0.0
q4-3s	0.14	4790.5	5209.0	-5209.0	0.0
q4-60s	0.22	9.5	4927.7	-2463.6	0.0
q5-3s	0.18	3308.6	3580.8	-3580.8	0.0
q5-60s	0.25	9.8	4133.4	-2066.4	0.0

3.2. Presión sintética: capacidad y política

Corridas válidas disponibles: 12.

Caso	Hits	Misses	Hit rate	p95 ms	Evict.	Exp.
U-10mb-lfu	82	918	0.082	10.9	805	0
U-10mb-lru	87	913	0.087	11.7	800	0
U-2mb-lfu	7	993	0.007	10.9	983	0
U-2mb-lru	8	992	0.008	11.0	981	0
U-5mb-lfu	40	960	0.040	10.3	911	0
U-5mb-lru	39	961	0.039	10.9	912	0
Z-10mb-lfu	665	335	0.665	10.2	222	0
Z-10mb-lru	643	357	0.643	10.3	244	0
Z-2mb-lfu	403	597	0.403	10.0	587	0
Z-2mb-lru	263	737	0.263	10.5	727	0
Z-5mb-lfu	584	416	0.584	10.9	367	0
Z-5mb-lru	509	491	0.509	10.1	443	0

Caso	Resp./s	p50 ms	Origen ms	Efic. ms	Evict./min
U-10mb-lfu	111.19	8.6	N/A	N/A	5370.7
U-10mb-lru	104.50	9.1	N/A	N/A	5015.9
U-2mb-lfu	109.08	8.8	N/A	N/A	6433.3
U-2mb-lru	109.37	8.7	N/A	N/A	6437.3
U-5mb-lfu	111.82	8.6	N/A	N/A	6111.9
U-5mb-lru	108.47	8.8	N/A	N/A	5935.5
Z-10mb-lfu	113.10	8.5	N/A	N/A	1506.5
Z-10mb-lru	112.95	8.5	N/A	N/A	1653.5
Z-2mb-lfu	113.17	8.5	N/A	N/A	3985.7
Z-2mb-lru	111.43	8.6	N/A	N/A	4860.8
Z-5mb-lfu	109.64	8.7	N/A	N/A	2414.3
Z-5mb-lru	112.80	8.5	N/A	N/A	2998.4

3.3. Consultas reales: Uniforme y Zipf

Corridas válidas disponibles: 8.

Caso	Hits	Misses	Hit rate	p95 ms	Evict.	Exp.
U-10mb-lru	14	26	0.350	4839.4	0	0
U-2mb-lru	14	26	0.350	4882.5	0	0
U-5mb-lfu	14	26	0.350	5274.4	0	0
U-5mb-lru	14	26	0.350	4881.4	0	0
Z-10mb-lru	25	15	0.625	4805.6	0	0
Z-2mb-lru	25	15	0.625	4696.0	0	0
Z-5mb-lfu	25	15	0.625	4754.5	0	0
Z-5mb-lru	25	15	0.625	4746.6	0	0

Caso	Resp./s	p50 ms	Origen ms	Efic. ms	Evict./min
U-10mb-lru	0.38	3299.0	3998.8	-2599.0	0.0
U-2mb-lru	0.41	3147.0	3705.0	-2408.0	0.0
U-5mb-lfu	0.39	3248.0	3876.7	-2519.6	0.0

Caso	Resp./s	p50 ms	Origen ms	Efic. ms	Evict./min
U-5mb-lru	0.39	3194.3	3890.0	-2528.3	0.0
Z-10mb-lru	0.67	10.7	3790.3	-1421.0	0.0
Z-2mb-lru	0.69	10.3	3703.5	-1388.5	0.0
Z-5mb-lfu	0.68	10.3	3741.2	-1402.6	0.0
Z-5mb-lru	0.68	9.5	3771.8	-1414.1	0.0

3.4. Origen real precargado: procesamiento en memoria

Corridas válidas disponibles: 1.

Caso	Hits	Misses	Hit rate	p95 ms	Evict.	Exp.
5mb-lru	14	26	0.350	446.3	0	0

Caso	Resp./s	p50 ms	Origen ms	Efic. ms	Evict./min
5mb-lru	3.44	310.9	360.7	-234.2	0.0

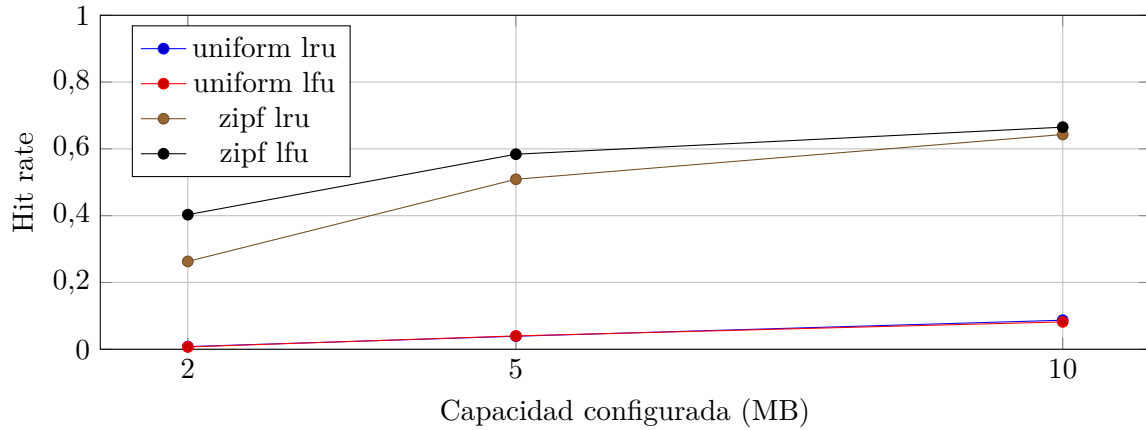


Figura 1: Presión sintética: no incluye latencia de Soccerway.

4. Comparaciones cuantitativas

pressure, uniform, LRU: al pasar de 2 a 10 MB, hit rate 0.008 a 0.008; evictions 981 a 800; p95 11.0 a 11.7 ms.

pressure, zipf, LRU: al pasar de 2 a 10 MB, hit rate 0.263 a 0.643; evictions 727 a 244; p95 10.5 a 10.3 ms.

pressure, 2 MB, LRU: Zipf cambia el hit rate en +25.5 puntos porcentuales frente a Uniforme; evictions 727 frente a 981; throughput 111.43 frente a 109.37 respuestas/s.

pressure, 5 MB, LRU: Zipf cambia el hit rate en +47.0 puntos porcentuales frente a Uniforme; evictions 443 frente a 912; throughput 112.80 frente a 108.47 respuestas/s.

pressure, 10 MB, LRU: Zipf cambia el hit rate en +55.6 puntos porcentuales frente a Uniforme; evictions 244 frente a 800; throughput 112.95 frente a 104.50 respuestas/s.

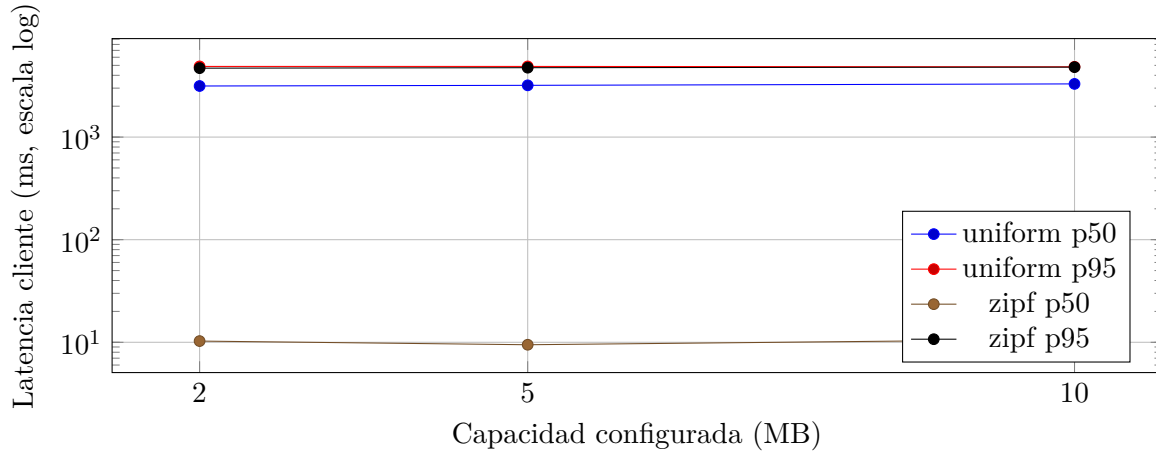


Figura 2: Consultas reales con LRU: mediana y cola de latencia.

pressure, 2 MB, LFU: Zipf cambia el hit rate en +39.6 puntos porcentuales frente a Uniforme; evictions 587 frente a 983; throughput 113.17 frente a 109.08 respuestas/s.

pressure, 5 MB, LFU: Zipf cambia el hit rate en +54.4 puntos porcentuales frente a Uniforme; evictions 367 frente a 911; throughput 109.64 frente a 111.82 respuestas/s.

pressure, 10 MB, LFU: Zipf cambia el hit rate en +58.3 puntos porcentuales frente a Uniforme; evictions 222 frente a 805; throughput 113.10 frente a 111.19 respuestas/s.

functional, uniform, LRU: al pasar de 2 a 10 MB, hit rate 0.350 a 0.350; evictions 0 a 0; p95 4882.5 a 4839.4 ms.

functional, zipf, LRU: al pasar de 2 a 10 MB, hit rate 0.625 a 0.625; evictions 0 a 0; p95 4696.0 a 4805.6 ms.

functional, 2 MB, LRU: Zipf cambia el hit rate en +27.5 puntos porcentuales frente a Uniforme; evictions 0 frente a 0; throughput 0.69 frente a 0.41 respuestas/s.

functional, 5 MB, LRU: Zipf cambia el hit rate en +27.5 puntos porcentuales frente a Uniforme; evictions 0 frente a 0; throughput 0.68 frente a 0.39 respuestas/s.

functional, 10 MB, LRU: Zipf cambia el hit rate en +27.5 puntos porcentuales frente a Uniforme; evictions 0 frente a 0; throughput 0.67 frente a 0.38 respuestas/s.

functional, 5 MB, LFU: Zipf cambia el hit rate en +27.5 puntos porcentuales frente a Uniforme; evictions 0 frente a 0; throughput 0.68 frente a 0.39 respuestas/s.

En las consultas reales, cuando los hits superan la mitad de las solicitudes, p50 puede pasar a reflejar el camino rápido de caché. Si más del 5 % siguen siendo misses, p95 aún puede reflejar el origen lento: mejorar la mediana no garantiza eliminar la cola. Los intervalos entre consultas afectan throughput, pero no se incluyen dentro de la latencia individual de respuesta.

5. Integración y fallos controlados

Se probaron servicios Docker reales, usando el origen precargado para no depender de la red externa.

Caso	HTTP esperado	HTTP observado	Segundos
Consulta inválida	400	400	0.01
Primer acceso	200	200	0.16
Repetición	200	200	0.03
Scraper detenido	502	502	8.02
Redis detenido	503	503	8.02
Métricas detenidas	200	200	8.01

Todas las comprobaciones pasaron.

Los errores de consulta inválida, scraper detenido y Redis detenido se registran cuando Metrics está disponible. Si Metrics está detenido, el hit sigue funcionando pero ese evento no puede persistirse: la telemetría es best-effort, sin cola durable. El runner detecta pérdida de eventos y no valida la corrida.

El cliente Redis limita reintentos y tiempos de socket; si Redis ya falló, no se vuelve a consultarlo para registrar estadísticas del error. Los tiempos de socket no son una garantía universal de duración de DNS; se reportan los tiempos observados en este entorno.

6. Preguntas y respuestas

¿Cómo cambian hit rate y miss rate entre distribuciones? Las comparaciones cuantitativas anteriores indican la diferencia medida; con cero errores, miss rate es uno menos hit rate. No debe confundirse porcentaje de hits sobre consultas válidas con porcentaje sobre todos los intentos cuando hay errores.

¿Qué distribución beneficia o estresa más la caché y por qué? Bajo presión, Uniforme reparte accesos entre más claves y provoca más reemplazos; Zipf reutiliza las populares. La tabla permite comprobar si esta hipótesis se cumple en cada capacidad y política. Un efecto pequeño o nulo entre políticas cuando los datos caben no demuestra equivalencia bajo cualquier carga.

¿Qué implicaciones tiene para una plataforma deportiva real? Eventos de alta audiencia concentran accesos a ciertos equipos y resultados. Reutilizar esos datos puede reducir llamadas al origen, pero exige TTL que limite obsolescencia. Los experimentos cerrados y sintéticos no prueban capacidad ante millones de usuarios: harían falta carga abierta concurrente, varias semillas y observación de saturación.

¿Qué sucede al aumentar memoria y cambiar política? El gráfico de presión muestra las tres capacidades y ambas políticas. Si se conserva el conjunto popular, LFU puede superar LRU; con accesos uniformes el beneficio por frecuencia es menor. Se reportan los resultados observados, no una superioridad universal. El aumento de memoria sólo produce más hits si antes se perdían entradas útiles por capacidad.

¿Cómo afecta TTL a Q1–Q5? La tabla TTL comprueba cada consulta individualmente. TTL 3 s vence antes de repetir a los 4 s; TTL 60 s conserva la respuesta. Frescura y rendimiento son objetivos distintos: estas pruebas verifican mecanismo, no si una respuesta quedó obsoleta en el sitio.

7. Análisis crítico y limitaciones

Las pruebas TTL con dos repeticiones aíslan expiración: un intervalo de 4 s supera TTL 3 s y produce miss seguido de miss; TTL 60 s conserva la respuesta y produce miss seguido de hit. Una

segunda expiración al final de una corrida puede aparecer en el contador sin significar un tercer acceso. Los percentiles de dos solicitudes no son estimaciones robustas de rendimiento.

La concentración Zipf permite reutilizar un subconjunto popular; Uniforme dispersa accesos entre más claves y suele generar más misses bajo capacidad limitada. La dirección y magnitud deben leerse en las tablas, no suponerse como resultado universal. Aumentar memoria reduce reemplazos cuando el conjunto activo supera la capacidad; si las respuestas reales ya caben, ampliar de 2 a 10 MB puede no cambiar hit rate ni producir evictions. La comparación LRU/LFU se realiza con idéntico catálogo y semilla, pero el tiempo interno de Redis y sus algoritmos aproximados pueden introducir variación.

El catálogo no contiene el mismo número de claves de cada tipo: Uniforme significa uniforme entre consultas completas, no entre las cinco categorías. Zipf cambia también la mezcla de tipos y parámetros, cuyos tiempos de origen pueden ser diferentes; por eso las diferencias de throughput real no se atribuyen exclusivamente a hit rate. La matriz sintética controla tamaño de valor y elimina esa heterogeneidad, pero pierde realismo. Las curvas de presión incluyen calentamiento desde caché vacía y no sólo estado estacionario.

La latencia real también depende de Soccerway, Chromium y red; no es atribuible sólo a Redis. En ciclo cerrado, misses lentos reducen throughput. Q3 depende de hallar una referencia de partido; ausencia de referencia devuelve error explícito y no prueba que nunca hubo enfrentamientos. La extracción depende del diseño de la fuente y puede romperse si cambia. No se implementan colas ni dashboard avanzado, correspondientes a futuras entregas según la guía.

Como criterio de frescura, Q1 y Q5 merecen TTL más corto cuando hay partidos en curso o cambios de calendario; Q2 y Q3 contienen principalmente historia y admiten TTL mayor; Q4 depende de si el intervalo incluye fechas actuales. Son decisiones propuestas, no conclusiones de obsolescencia comprobada: las pruebas sólo verifican expiración y reutilización. La implementación permite variar TTL entre corridas, pero usa un TTL global, no cinco TTL simultáneos. Los valores 3 y 60 s son extremos experimentales, no una recomendación de producción.

Las fechas de la fuente sin año usan el año de obtención del snapshot o el actual en vivo; no se reasignan al año del intervalo solicitado. La vista actual no equivale a un archivo completo de temporadas históricas. Veintiuna pruebas automatizadas comprueban fórmulas, fechas, claves, distribución y fallos simulados; las 54 consultas sobre precarga se verifican con el acceso externo bloqueado.

8. Conclusiones

Con datos reales y 5 MB LRU, hit rate fue 35.0 % con Uniforme y 62.5 % con Zipf. La diferencia describe esta muestra; no garantiza el mismo resultado con otros catálogos o semillas.

Presión Zipf 2 MB: LFU cambia hit rate en +14.0 puntos frente a LRU. La frecuencia ayuda a conservar claves populares bajo reemplazos; cuando aumenta la capacidad, ambas pueden retener más del conjunto activo.

Presión Zipf 5 MB: LFU cambia hit rate en +7.5 puntos frente a LRU. La frecuencia ayuda a conservar claves populares bajo reemplazos; cuando aumenta la capacidad, ambas pueden retener más del conjunto activo.

Presión Zipf 10 MB: LFU cambia hit rate en +2.2 puntos frente a LRU. La frecuencia ayuda a conservar claves populares bajo reemplazos; cuando aumenta la capacidad, ambas pueden retener más del conjunto activo.

Las corridas reales y las sintéticas responden preguntas diferentes: si los datos reales caben, más memoria no necesariamente mejora hits; bajo presión, capacidad y política sí influyen en permanencia. TTL se verificó individualmente para Q1–Q5 y debe elegirse también por frescura. Mejorar p50 no garantiza mejorar p95. Antes de extrapolar a alta audiencia se requieren más muestras, distintas semillas y carga concurrente abierta.

9. Reproducción y entrega

Desde Windows con Python y Docker Desktop, ejecutar secuencialmente `python experiments/run_suite.py ttl`, luego `pressure` y `functional`; ejecutar `python experiments/analyze_results.py` y `python experiments/build_report.py`. Las pruebas automatizadas están en `tests/run_tests.ps1`. README documenta contratos, puertos, modo continuo y parada con Ctrl+C.

Repositorio de entrega: <https://giteit.udp.cl/CIT2011/2026-2/seccion-2/grupo-4/-/tree/TheRealMain>. Video: en preparación por los integrantes. Debe añadirse un enlace real de YouTube o Drive visible para el docente. Confirmar plazo y referencias inconsistentes de semestre con el docente; enviar PDF y enlace del repositorio por Canvas.

10. Fuentes de evidencia

Guía *Tarea 1 Sistemas Distribuidos 2026 2* y *Rúbrica T1 SD 2026 2*, suministradas por los integrantes. Datos reales: Soccerway, obtenidos en la fecha de las corridas, sin garantía de frescura posterior. Evidencia reproducible: `experiments/results/`, esquema 2; resumen `experiments/analysis/summary.csv`, verificación `validation.json`. Los archivos históricos de esquema anterior se excluyen. Cada JSON incluye configuración, semilla, revisión Git, marca de cambios locales, eventos, muestras funcionales y contadores Redis.